CMPM 121 Assignment 3 Report

Architecture Diagram

```
classDiagram
    GameManager --> RewardScreenManager : stores pending reward spell
    RewardScreenManager --> SpellBuilder : generates wave reward
    RewardScreenManager --> PlayerController : reads spell inventory
    EnemySpawner --> PlayerController : starts waves and accepts rewards
    PlayerController --> SpellCaster : owns mana and spell slots
    PlayerController --> Hittable : scales max HP by wave
    PlayerController --> ClassesJson : evaluates class stats
    SpellCaster --> Spell : casts selected spell
    SpellCaster --> SpellBuilder : creates starting spell
    SpellUIContainer --> SpellCaster : shows four slots
    SpellUIContainer --> SpellUI : refreshes slot views
    SpellUI --> SpellCaster : checks drop state
    SpellBuilder --> SpellsJson : loads base and modifier definitions
    SpellBuilder --> SpellDefinition : stores JSON attributes
    Spell --> SpellCastProfile : builds cast-time values
    Spell --> ProjectileSpellData : configures projectile movement
    Spell --> ProjectileManager : creates projectiles
    Spell --> Damage : applies on-hit damage

    class GameManager {
        +Spell pendingSpellReward
        +int pendingSpellRewardWave
        +void SetPendingSpellReward(Spell spell, int rewardWave)
        +void ClearPendingSpellReward()
    }

    class RewardScreenManager {
        +GameObject rewardUI
        -TextMeshProUGUI buttonText
        -TextMeshProUGUI messageText
        -SpellBuilder spellBuilder
        +void Start()
        +void Update()
        -PlayerController GetPlayer()
        -void EnsurePendingRewardSpell(PlayerController player)
        -string GetMessage(PlayerController player)
        -string GetRewardButtonText(PlayerController player)
        -string GetRewardDescription(PlayerController player)
        -string GetRewardSlotDescription(PlayerController player)
    }

    class SpellCaster {
        +int max_spells
        +int mana
        +int max_mana
        +int mana_reg
        +int spell_power
        +Spell spell
        +int SelectedIndex
        +int SlotCount
        +IEnumerator ManaRegeneration()
        +IEnumerator Cast(Vector3 where, Vector3 target)
        +Spell GetCurrentSpell()
        +Spell GetSpell(int slot)
        +int GetEquippedSpellCount()
        +bool CanDropSpell(int slot)
        +int GetEquipSlotForNextSpell()
        +bool SelectSpell(int slot)
        +bool EquipSpell(Spell nextSpell)
        +bool EquipSpellAt(Spell nextSpell, int slot)
        +bool DropSpell(int slot)
```

```
}

class SpellBuilder {
    -Dictionary~string, SpellDefinition~ definitions
    -List~SpellDefinition~ baseSpells
    -List~SpellDefinition~ modifierSpells
    +Spell Build(SpellCaster owner)
    +Spell BuildRandom(SpellCaster owner, int maxModifiers)
    +Spell BuildReward(SpellCaster owner, int wave)
    +Spell BuildWithModifiers(SpellCaster owner, string baseId, string[] modifierIds)
    -void LoadDefinitions()
    -string LoadSpellData()
    -SpellDefinition GetDefinition(string id)
}

class SpellDefinition {
    +string id
    +JObject attributes
    +bool IsBaseSpell()
    +string GetString(string key, string fallback)
    +JObject GetObject(string key)
    +int EvaluateInt(string key, SpellCaster owner, int fallback)
    +float EvaluateFloat(string key, SpellCaster owner, float fallback)
    +static int EvaluateInt(JToken token, SpellCaster owner, int fallback)
    +static float EvaluateFloat(JToken token, SpellCaster owner, float fallback)
}

class Spell {
    +float last_cast
    +SpellCaster owner
    +Hittable.Team team
    +string GetName()
    +int GetManaCost()
    +int GetDamage()
    +string GetDescription()
    +float GetCooldown()
    +int GetIcon()
    +string GetBaseId()
    +List~string~ GetModifierNames()
    +bool IsReady()
    +IEnumerator Cast(Vector3 where, Vector3 target, Hittable.Team team)
    -SpellCastProfile BuildProfile()
    -void ApplyModifiers(SpellCastProfile profile)
    -IEnumerator CastVolley(Vector3 where, Vector3 target, Hittable.Team hitTeam, SpellCastProfile pr
    -void CastProjectiles(Vector3 where, Vector3 direction, Hittable.Team hitTeam, SpellCastProfile p
    -void OnHit(Hittable other, Vector3 impact, Hittable.Team hitTeam, SpellCastProfile profile)
    -void CastSecondaryProjectiles(Vector3 impact, Hittable.Team hitTeam, SpellCastProfile profile)
    -void CastOnHitProjectiles(Vector3 impact, Hittable.Team hitTeam, SpellCastProfile profile)
}

class SpellUIContainer {
    +GameObject[] spellUIs
    +PlayerController player
    +void SelectSlot(int slot)
    +void DropSlot(int slot)
}

class SpellUI {
    +GameObject icon
    +RectTransform cooldown
    +TextMeshProUGUI manacost
    +TextMeshProUGUI damage
    +GameObject highlight
    +GameObject dropbutton
    +int slotIndex
    +PlayerController player
    +void Setup(SpellUIContainer container, PlayerController player, int slotIndex)
    +void SetSpell(Spell spell)
    +void DropSpell()
}
```

Architecture Description

Spell data is loaded from ‘Assets/Resources/spells.json‘ into ‘SpellDefinition‘ objects. ‘SpellBuilder‘ separates base spells from modifier spells, builds the player’s starting Arcane Bolt, and generates random reward spells by combining one base spell with a random list of modifier definitions.

‘Spell‘ is a data-driven runtime spell rather than a MonoBehaviour. When cast, it builds a ‘SpellCastProfile‘ from the base definition, evaluates RPN expressions with the current ‘wave‘ and player ‘power‘, then applies each modifier in order. The profile controls damage, mana cost, cooldown, projectile speed, projectile trajectory, repeated casts, split volleys, secondary projectiles, and custom on-hit projectiles.

The player now owns a ‘SpellCaster‘ with up to four spell slots. ‘SpellUIContainer‘ keeps the four slot UIs visible and synchronized with the inventory, while ‘SpellUI‘ shows each slot’s icon, mana cost, damage, cooldown, highlight state, and drop button. The reward screen stores a pending reward in ‘GameManager‘, previews its stats, and shows the slot where the reward will be equipped or the spell it will replace. ‘EnemySpawner.NextWave‘ accepts the pending reward before starting the next wave.

Player progression is loaded from ‘classes.json‘. ‘PlayerController.ApplyWaveStats‘ evaluates the mage class expressions for HP, mana, mana regeneration, spell power, and speed each wave, preserving current HP and mana percentages when the maximum values change.

New Spells

- ‘damage_amp‘: increases spell damage and mana cost multiplicatively.
- ‘speed_amp‘: increases projectile speed multiplicatively.
- ‘doubler‘: casts the modified spell a second time after a short delay, with higher mana cost and cooldown.
- ‘splitter‘: casts the modified spell in two nearby directions.
- ‘chaos‘: increases damage and changes the projectile trajectory to spiraling.
- ‘homing‘: changes the projectile trajectory to homing while reducing damage and adding mana cost.
- ‘mana_stream‘: an added modifier that lowers mana cost with a small cooldown penalty.
- ‘overclocked‘: an added modifier that reduces cooldown and increases projectile speed, but costs more mana.
- ‘nova_burst‘: an added behavior modifier that releases extra arcane shards from the hit point.

Optional base spells from the assignment are also supported: Magic Missile uses homing projectiles, Arcane Spray creates a cone of short-lived bolts, and Arcane Blast creates secondary projectiles when it hits.

Contributions

Todd Crandell implemented the JSON-driven spell definition pipeline, modifier evaluation, reward generation, player stat scaling, and the final reward-slot/drop polish. The main lesson was how much cleaner spell behavior becomes when the cast path builds a temporary profile from data instead of baking every variation into separate classes.

ValerieVATS expanded the player spell system from a single spell into multiple spell slots and added the visible slot UI with selection and drop affordances. The main lesson was

keeping gameplay state and UI state synchronized without letting the UI become the source of truth.